1 Evaluation Methodology

This chapter details the methodology used to evaluate the **Agentic Project Management (APM)** framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants.

The evaluation was designed to measure three distinct dimensions:

- **Economic Efficiency** *[cost framing pending lit review]*
- **Process Efficiency** (human cognitive load and interaction burden)
- **Functional Efficacy** (ability to complete non-trivial tasks)

[Unified study design — single section describing the Copilot-based comparative study. To be written in Phase 3.]

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM *[phase names pending APM v1 terminology update in Phase 2]* yields a tangible return on investment compared to a single-agent workflow.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):** *[cost framing pending lit review — to be reframed around request consumption / resource efficiency rather than raw token cost]*
- **RQ2 (Process Efficiency):**
  - Does the structured coordination of APM reduce the "human burden" during execution? Specifically, does it reduce the need for user interventions, corrections, and clarifications?
  - Is the reduction in human burden due to the structured coordination of APM significant enough to justify the structural overhead?
  - Are the agents able to effectively coordinate and complete the task without the need for constant human oversight?
- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where unstructured agentic workflows typically encounter "Agentic Handoff Failures"?
  - How does the performance of APM compare to the performance of a single-agent workflow?

1.2 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`, a Python-based Model Context Protocol (MCP) server designed to interface with the `openeclass` platform. Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

1.2.1 The Task:

The final task requires participants to implement a new `get_course_announcements` tool in the Python MCP server and, crucially, to create a complete native C replica of the MCP server. The task specification was refined through iteration to ensure sufficient complexity for differentiating multi-agent from single-agent approaches.[1]

This C implementation adds significant complexity, turning the assignment into a challenge of managing code translation, memory safety, and protocol details across two very different languages. As such, the task is specifically designed to stress-test the limits of agent coordination and reveal differences between multi-agent and single-agent approaches, especially in scenarios where context management becomes a bottleneck.

Key Technical Challenges:

1. **Dynamic Data Source:** The required data for implementing the new feature is not available in static HTML for parsing. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.
2. **Nested Parsing:** The server returns JSON where the payload is an array of raw HTML strings, requiring a second layer of parsing to extract the data.
3. **Client-Side Filtering:** The legacy API does not support date filtering. The agent must implement robust client-side logic to parse and filter unique locale-formatted dates.
4. **C Implementation:** Agents must build a native C MCP server replica, manually implementing JSON-RPC 2.0 with standard C string manipulation (no external JSON libraries such as cJSON/Jansson). Use of `libcurl` for HTTP/HTTPS and `libxml2` for HTML parsing, was required for the eclass MCP server functionality.
5. **Memory Safety:** The C implementation must be memory-safe, passing Valgrind checks without leaks or errors.

Deliverables:

1. Core server logic in modular Python files (existing and new functionality).
2. A native C implementation of the MCP server with equivalent (existing and new) functionality.
3. A working build system (Makefile) for the C replica.

[1] Task specification and grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task

1.2.2 Grading:

Submissions are evaluated using an automated black-box test suite with comprehensive validation, grading, metric extraction, and dataset aggregation capabilities (see the evaluation appendix for detailed infrastructure documentation).[2]

The test suite awards **48 points total**: 12 points for baseline functionality (existing Python tools), 12 points for the new Python feature (`get_course_announcements`), and 24 points for the complete C replica implementation. A submission with no patch applied receives the baseline score of **12/48 points**.

All tests are run against fixed baseline commits to ensure reproducibility: `eclass-mcp-server` at commit `8b4e39c` and `openeclass` at commit `e8b3329` (Docker image `openeclass:test`).

1.2.3 Repository Architecture and Data Management

The case study employs a two-repository system separating public resources from sensitive participant data. The **task repository** (`bsc-apm-case-study-task`) is openly accessible and includes task specifications, grading infrastructure, and participant guides. The **dataset repository** (`bsc-apm-case-study-data`) contains all submitted participant data: code patches, sanitized chat transcripts (see Section 1.2.4), and anonymized session metadata.

Access to the dataset repository's secret access key requires explicit authorization and compliance with data usage agreements. This controlled access mechanism enables peer review and reproducibility while maintaining participant privacy protections.

1.2.4 Sanitization and PII Handling

Chat exports contain sensitive information (unstructured user-input text, file paths, command histories, timestamps). To protect participant privacy while enabling reproducibility, all exports are sanitized using reversible encryption before storage.

Each chat is assigned a unique identifier via SHA256 hashing of concatenated user messages for anonymous cataloging. Essential metadata (chat ID, model IDs, request count) is preserved in plaintext to enable dataset organization without decryption, while conversation content is encrypted using an HMAC-SHA256 XOR cipher. The same sanitization method is applied uniformly across both platforms.

For detailed technical specifications of the anonymization and encryption processes, see the evaluation appendix.

[2] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

1.3 Study Design

[Unified single-study design — to be written in Phase 3. Will describe the Copilot-based comparative evaluation: data source (Copilot JSON chat exports), participant pool (University students with GitHub Education Pack or premium subscription), two-session within-subjects design with counterbalanced framework order, and the cost framing (pending lit review).]

1.4 Frameworks Under Evaluation

Two workflow configurations are evaluated:

1. **Agentic Project Management (APM v1):** The proposed framework. *[APM v1 description pending Phase 3 rewrite — to be grounded in apm-website/docs/. Will cover the multi-agent, spec-driven architecture (Planner/Manager/Worker), the Planning and Implementation phases, and file-based coordination via the Message Bus.]*

2. **GitHub Spec-kit:** A structured, spec-driven baseline. It utilizes a single-agent workflow guided by rigid templates and three sequential commands (`specify`, `plan`, `tasks`).

1.5 Metrics and Data Collection

[Metrics section pending Phase 4 redesign and the separate cost literature search effort. The final metric set will cover: Session Metadata, Cost Metrics (request consumption as proxy for enterprise budget impact), Reliability Metrics (tool success/retry, user edit rate), Process Metrics (behavior distribution and magnitude, session duration), and Functional Metrics (the 48-point grading rubric).]

2 Results

2.1 Functional Performance Analysis

Primary Metrics: Total score distribution, completion rates (48/48, 24-47, 12-23, baseline-only), baseline regression rates (python_baseline_score < 12), feature implementation success (Python feature and C replica completion rates).
Category Analysis: Success rates per test category (build, protocol, login, logout, courses, authstatus, announcements) for both Python and C implementations. Identifies framework-specific failure patterns and strengths.
Comparative Analysis: T-tests comparing APM vs. Spec-kit for total score, category scores, and completion rates. Effect sizes (Cohen's d) to assess practical significance.

2.2 Economic and Process Efficiency Analysis

[Pending Phase 5 rewrite — single unified analysis section (no Study A/B split). Will reference the request-based cost framing and the redesigned process metrics once the lit review lands.]

2.3 Learning Effects and Confounding

Critical Issue: Participants complete the same task twice (Session 1 and Session 2) using different frameworks. This creates a learning effect where task familiarity from S1 influences S2 performance, potentially confounding framework comparisons.
Analysis Strategy:

- **Learning Effect Matrix:** Calculate mean improvement (percentage change) from S1 to S2 for both S1→S2 framework combinations across all metrics (functional scores, cost, process efficiency)
- **Order Effect:** Compare APM→Spec-kit vs. Spec-kit→APM directions. Tests whether order of framework exposure affects learning.
- **Baseline Learning:** Overall mean improvement across both directions (pure task learning effect regardless of framework)

Key Questions: Does task learning confound framework comparisons? Is there a "best first framework" that maximizes S2 improvement? How do we control for learning effects when comparing frameworks?

3 Discussion

3.1 Cost-Benefit Analysis: Context Overhead vs. Failure Prevention

[Pending Phase 5 rewrite — to be reframed around request consumption overhead vs failure prevention, once the lit review establishes the cost framing.]

3.2 Process Efficiency: Protocol Overhead vs. Intervention Reduction

[Pending Phase 5 rewrite — to be updated for APM v1 terminology and the two-framework comparison.]

3.3 Functional Efficacy: Reliability and Failure Patterns

[Pending Phase 5 rewrite — to be updated for the two-framework comparison.]

3.4 Learning Effects: Confounding and Framework Transferability

[Pending Phase 5 rewrite — simplified for the two-framework design (APM v1 vs Spec-kit, in both orders).]

3.5 Threats to Validity

[Pending Phase 5 rewrite — Cursor/model-standardization threats removed; new threats relevant to a single-platform, two-framework study to be added.]